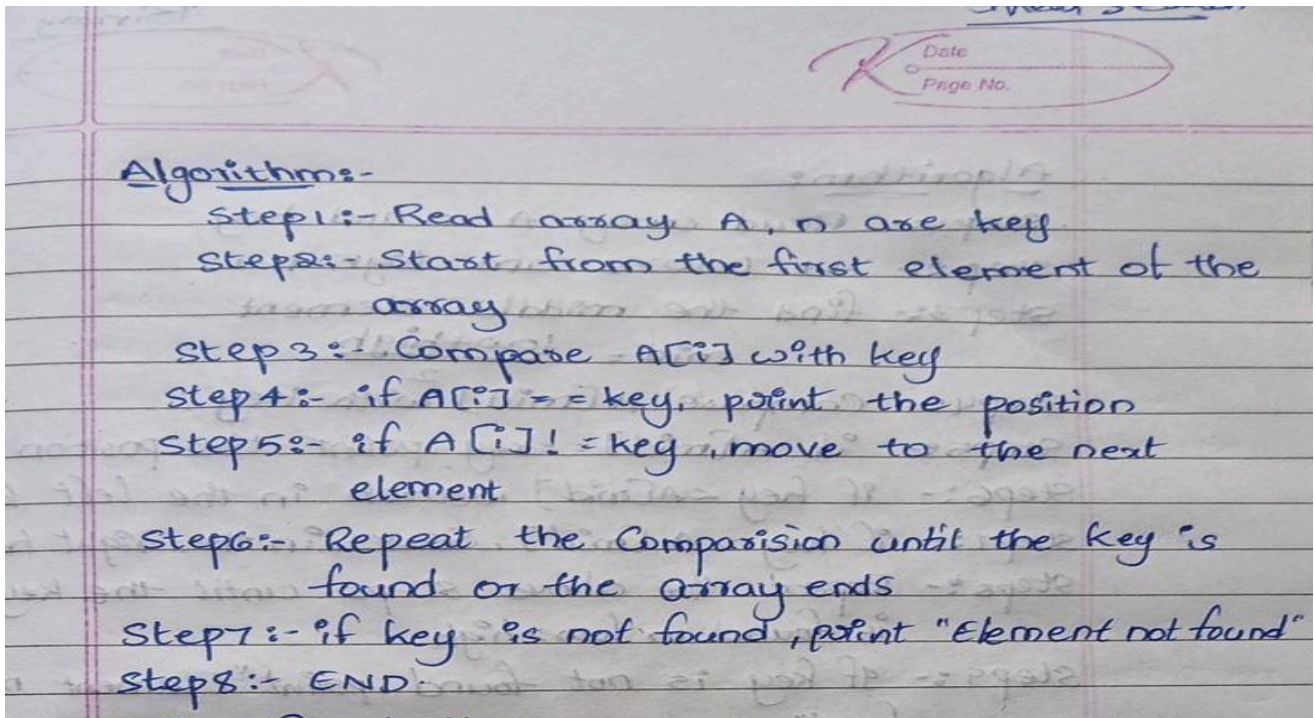


Practical: 2

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

1) Linear search

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
```

```
int arr[n];    cout << "Enter " << n
<< " elements: ";
for (int i = 0; i < n; i++) {
cin >> arr[i];
}

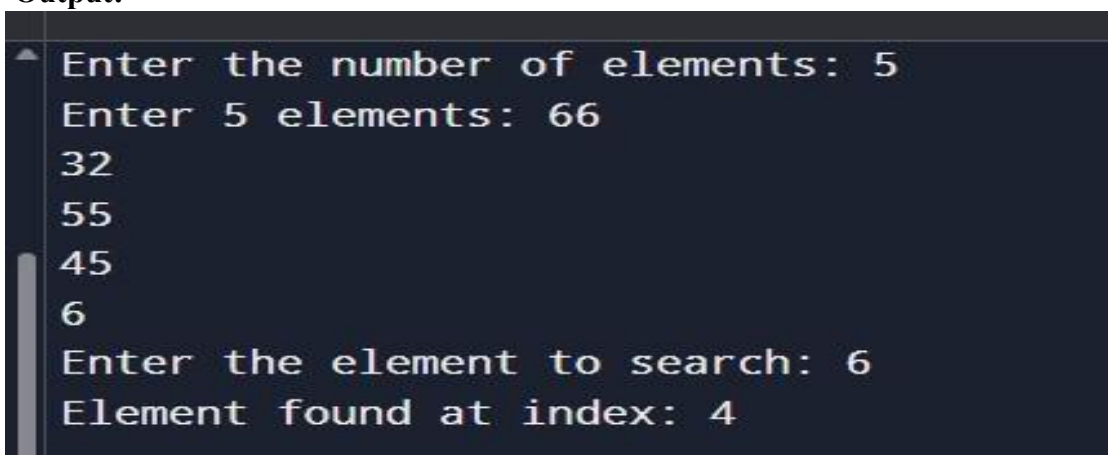
int target;    cout << "Enter the element
to search: ";
cin >> target;

int index = -1; for (int i
= 0; i < n; i++) {    if
(arr[i] == target) {
index = i;          break;
}
}

if (index != -1) {    cout << "Element found at
index: " << index << endl;
} else {
cout << "Element not found" << endl;
}

return 0;
}
```

Output:



```
Enter the number of elements: 5
Enter 5 elements: 66
32
55
45
6
Enter the element to search: 6
Element found at index: 4
```

Time Complexity:

Time Complexity:-

Best Case:-

Linear Search

(Best Case)

element is found at the first position

```
for (i = 0; i < n; i++) {
```

```
    if (arr[i] == key) {
```

```
        return i;
```

```
    }
```

```
return -1;
```

$T(n) = 1$

$= O(1)$

Average Case

Linear Search

```
for (i = 0; i < n; i++) {
```

```
    if (arr[i] == key) {
```

```
        return i;
```

```
    }
```

```
return -1;
```

3

3

```
return -1;
```

$T(n) = O(n)$

Worst Case

Linear Search

```
for (i = 0; i < n; i++) {
```

```
    if (arr[i] == key) {
```

```
        return i;
```

```
    }
```

```
return -1;
```

3

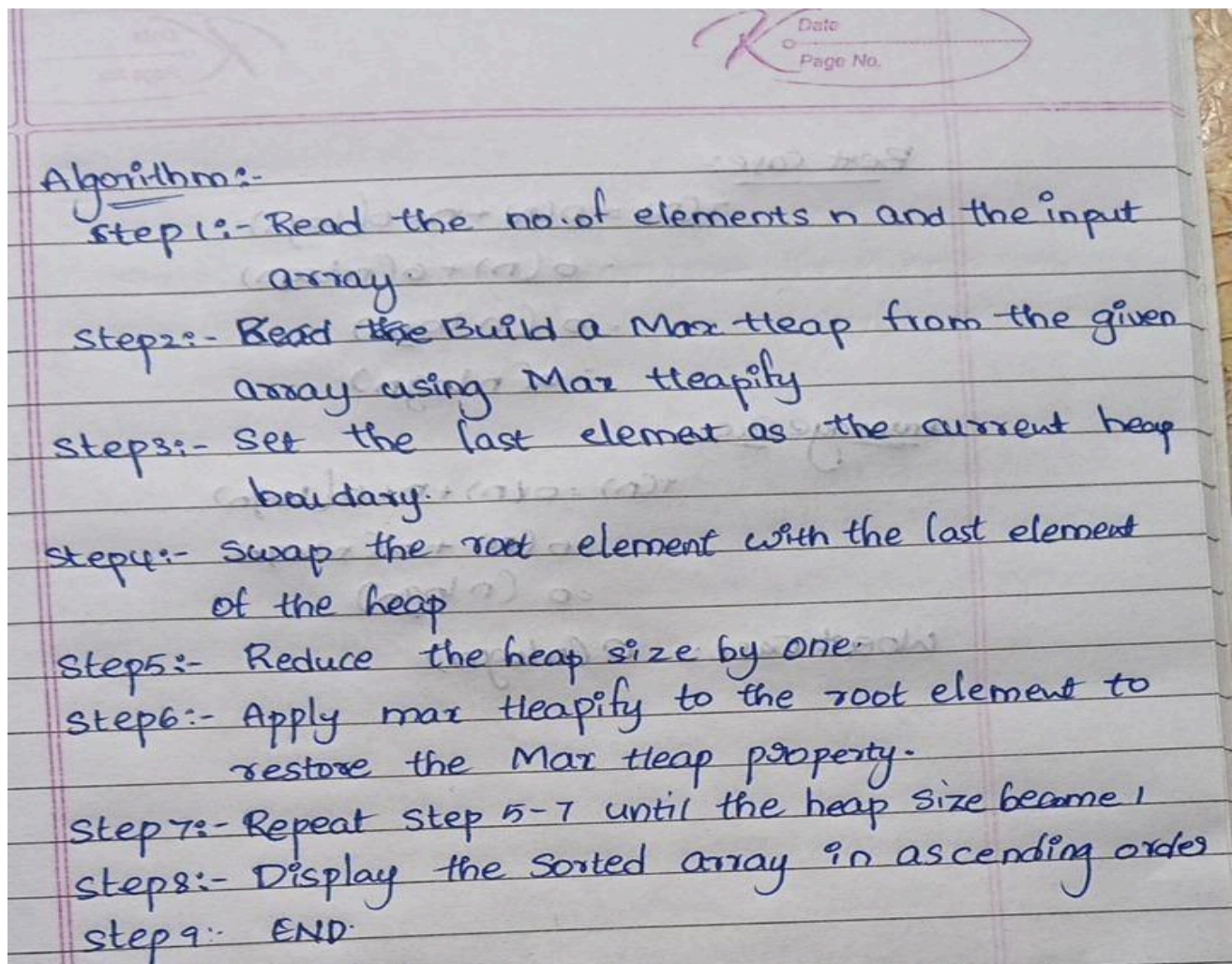
3

```
return -1;
```

$T(n) = O(n)$

2) Binary search

Algorithm:



Program code:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int binarySearch(const vector<int>& arr, int target) {  
    int low = 0;    int high = arr.size() - 1;
```

```
while (low <= high) {      int
mid = low + (high - low) / 2;

    if (arr[mid] == target) {
return mid;
    } else if (arr[mid] < target)
{ low = mid + 1;    } else {
high = mid - 1;
    }    }
return -1;
}

int main()
{
    int size;    cout << "Enter the number of
elements: ";
    cin >> size;

    vector<int> arr(size);    cout << "Enter " << size << "
elements in sorted order:\n";
    for (int i = 0; i < size; i++) {
cin >> arr[i];
    }

    int target;    cout << "Enter the element
to search for: ";
    cin >> target;

    int result = binarySearch(arr, target);

    if (result != -1) {        cout << "Element found at
index: " << result << endl;
    } else {
        cout << "Element not found in the array." << endl;
    }

    return 0;
}
```

Output:

Output

```
Enter the number of elements: 4
Enter 4 elements in sorted order:
12
36
48
52
Enter the element to search for: 52
Element found at index: 3
```

+

Time complexity:

Time Complexity:-
Best case:-
Binary search
(Best case)
 $mid = (low + high) / 2$
if $arr[mid] == key$
return mid
 $T(n) = 1$
 $= O(1)$
Average case:-
 $low = 0; -1$
 $high = n - 1; -1$
while $(low \leq high) \{ - O(\log n)$
 $mid = (low + high) / 2 - 1$
 if $(arr[mid] == key) - 1$
 { return mid; }
 else if $(arr[mid] < key) - 1$
 {
 $low = mid + 1; - 1$
 }
 else {
 $high = mid - 1 - 1$
 }
}
return -1; -1
 $T(n) = O(\log n)$

```

Worst Case-
low = 0; -1
high = n-1; -1
while (low <= high) {
    mid = (low+high)/2;
    if (arr[mid] == key) {
        return mid;
    }
    else if (arr[mid] < key) {
        low = mid+1;
    }
    else {
        high = mid-1;
    }
}
return -1;
T(n) = O(log n)
    
```

Conclusion:

The implementation and time analysis of Linear Search Binary Search helps us understand their efficiency. Linear Search is simple and can work on any array, but it may require $O(n)$ comparisons. Binary Search is faster with $O(\log n)$ time in average and worst cases, but it requires the array to be sorted. Therefore, Binary Search is more efficient for searching large sorted datasets.